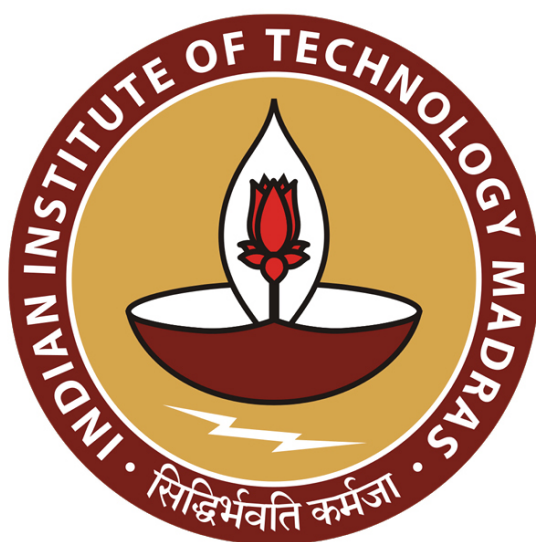




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pritam Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Relational Database Design/9: MVD and 4NF

Welcome to Module 29 of Database Management Systems course in the IIT Madras online BSc program.

(Refer Slide Time: 00:28)

The screenshot shows a presentation slide titled "Module Recap" from the IIT Madras online BSc program. The slide content includes a bullet point: "Using the specification for a Library Information System, we have illustrated how a schema can be designed and then refined for finalization". A small video inset shows Professor Partha Pritam Das. The slide footer includes "Database Management Systems", "Partha Pritam Das", and the slide number "12".

We have been discussing about Relational Database Design. We have done that through the functional dependency theory and normal forms and normalization and we culminated by discussing a specific Library Information System Design from specification to the relational schema, going through the application of each, I mean, different things that we have learned through the process. So, this brings us to kind of the learning for relational database design using functional dependency model.

(Refer Slide Time: 01:13)

IIT Madras
Module Objectives

- To understand multi-valued dependencies arising out of attributes that can have multiple values
- To define Fourth Normal Form and learn the decomposition algorithm to 4NF

Database Management Systems Partha Pratim Das

Now, we are going to generalize this somewhat to take care of more forms of redundancy that may exist in the real world and we are going to talk about fourth normal form.

(Refer Slide Time: 01:29)

IIT Madras
Module Outline

- Multivalued Dependencies
- Decomposition to 4NF

Database Management Systems Partha Pratim Das

And before that the multivalued dependencies.

(Refer Slide Time: 01:30)

Multivalued Dependency

Database Management Systems

So, like we had functional dependencies where a set of attributes values over a set of attributes uniquely determines the values over another set of attributes, alpha determines beta.

(Refer Slide Time: 01:47)

MVD: Multivalued Dependency

• Persons(Man, Phones, Dog_Like)

Man(M)	Phones(P)	Dogs_Like(D)
M1	P1/P2	D1/D2
M2	P3	D2

Key: MPD

Meaning of the tuples
Man M have phones P, and likes the dogs D.
M1 have phones P1 and P2, and likes the dogs D1 and D2.
M2 have phones P3, and likes the dog D2.

There are no non trivial FDs because all attributes are combined forming Candidate Key, that is, MDP. In the above relation, two multivalued dependencies exists:

- Man $\twoheadrightarrow$ Phones
- Man $\twoheadrightarrow$ Dogs_Like

A man's phone are independent of the dogs they like. But after converting the above relation in Single Valued Attribute, each of a man's phones appears with each of the dogs they like in all combinations.

Source: <http://www.edugrabs.com/multivalued-dependency-mvd/>

Post 1NF Normalization

Man(M)	Phones(P)	Dogs_Likes(D)
M1	P1	D1
M1	P2	D2
M2	P3	D2
M1	P1	D2
M1	P2	D1

Diagram: Man $\twoheadrightarrow$ Phones

MVD: Multivalued Dependency

• Persons(Man, Phones, Dog_Like)

Man(M)	Phones(P)	Dogs_Like(D)
M1	P1/P2	D1/D2
M2	P3	D2

Key: MPD

Meaning of the tuples
 Man M have phones P, and likes the dogs D.
 M1 have phones P1 and P2, and likes the dogs D1 and D2.
 M2 have phones P3, and likes the dog D2.

There are no non trivial FDs because all attributes are combined forming Candidate Key, that is, MDP. In the above relation, two multivalued dependencies exists:

- Man $\twoheadrightarrow$ Phones ✓
- Man $\twoheadrightarrow$ Dogs_Like ✓

A man's phone are independent of the dogs they like. But after converting the above relation in Single Valued Attribute, each of a man's phones appears with each of the dogs they like in all combinations.

Source: <http://www.edugrabs.com/multivalued-dependency-mvd/>

Handwritten notes: MVD, INF BCNF

Post 1NF Normalization

Man(M)	Phones(P)	Dogs_Likes(D)
M1	P1	D1
M1	P2	D2
M2	P3	D2

MVD: Multivalued Dependency

• Persons(Man, Phones, Dog_Like)

Man(M)	Phones(P)	Dogs_Like(D)
M1	P1/P2	D1/D2
M2	P3	D2

Key: MPD

Meaning of the tuples
 Man M have phones P, and likes the dogs D.
 M1 have phones P1 and P2, and likes the dogs D1 and D2.
 M2 have phones P3, and likes the dog D2.

There are no non trivial FDs because all attributes are combined forming Candidate Key, that is, MDP. In the above relation, two multivalued dependencies exists:

- Man $\twoheadrightarrow$ Phones
- Man $\twoheadrightarrow$ Dogs_Like

A man's phone are independent of the dogs they like. But after converting the above relation in Single Valued Attribute, each of a man's phones appears with each of the dogs they like in all combinations.

Source: <http://www.edugrabs.com/multivalued-dependency-mvd/>

Post 1NF Normalization

Man(M)	Phones(P)	Dogs_Likes(D)
M1	P1	D1
M1	P2	D2
M2	P3	D2

We have a more generalized scenario if we have multivalued attributes. So, far what we have dealt with we have enforced that we start with 1NF, atomic single value. So, 1NF if we start with one 1NF every attribute can have a single value, but what if it can have multiple values we saw this situation earlier too, but now, we are going to go in depth. So, we are talking about a relationship, a relational schema person who has name, phone and likeness for dogs.

Naturally the person has multiple phones a person can have multiple phones and likeness for multiple dogs. So, these are the multiple values that are possible for that attribute. Now, if they were not possible then we would have said that man functionally determines phones. But now, we cannot say that anymore.

So, what is we say we now model this saying that man multi determines mind you earlier it was functionally determines now it is multi determines phones and the change in notation is functional dependency was this and this dependency is written with two head arrows at times people also write it like this. So, I have we have here two multi dependencies here multivalued dependencies here.

Multivalued dependencies in like functional dependency we say FD, multivalued dependencies we often say it is MVD. Now, if you look into this person, relational schema from the perspective of FDs there is no trivial FD and what would be the key for this? It will be all the attributes. Because nothing functionally determines anything else. So, the candidate keys man doglike phone MDP.

So, that is, that being the only key and there not being any non-trivial functional dependency, this relationship is already in BCNF. But if you look into this that there are multiple values. So, this, this relationship, relational schema is not even in 1NF, kind of contradiction. So, to make it into 1NF, what you will have to do, you will have to flatten out the multiple values as you start doing here P1 P2, P1 P2, D1 D2, D2 D1.

So, you have all possible combinations P1 D1, P1 D2, P2 D1, P2 D2, because once it gets single valued and you have to keep these values it gets into 1NF it actually still does not have any non-trivial functional dependency. So, it is in BCNF but, you can see what a tremendous amount of redundancy it has now. So, that is the new problem which cannot be handled by functional dependencies and has to be handled by the multivalued dependencies that we are going to talk about today.

(Refer Slide Time: 06:08)

MVD (2)

- If two or more independent relations are kept in a single relation, then Multivalued Dependency is possible. For example, Let there are two relations :
 - **Student(SID, Sname) where $(SID \rightarrow Sname)$**
 - **Course(CID, Cname) where $(CID \rightarrow Cname)$**
- There is no relation defined between Student and Course. If we kept them in a single relation named **Student.Course**, then MVD will exist because of m:n Cardinality
- If two or more MVDs exist in a relation, then while converting into SVAs, MVD exists.

SID	Sname
S1	A
S2	B

CID	Cname
C1	C
C2	B

SID	Sname	CID	Cname
S1	A	C1	C
S1	A	C2	B
S2	B	C1	C
S2	B	C2	B

MVDs exist:
1. $SID \twoheadrightarrow CID$
2. $SID \twoheadrightarrow Cname$

Source: <http://www.edugrabs.com/multivalued-dependency-mvd/>

Now, if we have two independent relations, whenever we have two independent relations, say, we are talking about two independent relations here. One is student which is SID and Sname. Obvious functional dependency and other is course CID and Cname. If we have two independent relations, that is, I mean as such, there is nothing common between them I am nothing common in terms of attributes or any relationship.

But if we keep them in a single table let us say a student courses a student table is a course table, but if we keep them in a single table multivalued dependency will happen. Quite obviously, because, because since there is no relationship basically, when you keep them in the single table, what do you do? You do a join and in this joint, you just have a Cartesian product, there is no common attribute to filter. So, you get all of these and you get all this redundancy. And in this you have two multivalued dependencies SID multi determines CID and SID multi determines Cname. So, that is the.

(Refer Slide Time: 07:50)

MVD (3)

- Suppose we record names of children, and phone numbers for instructors:
 - $inst_child(ID, child_name)$
 - $inst_phone(ID, phone_number)$
- If we were to combine these schema to get
 - $inst_info(ID, child_name, phone_number)$ ✓
 - Example data:
 - (99999, David, 512-555-1234)
 - (99999, David, 512-555-4321)
 - (99999, William, 512-555-1234)
 - (99999, William, 512-555-4321)
- This relation is in BCNF
 - Why?

I mean this is just another perspective of the issue that example, I was trying to show. We talked about this example earlier also, this is instructor has multiple children, instructor's multiple phone numbers and keep them in different tables, but if you combine the schemas, if you have the instructors ID, you can go back to that module and see the details also.

If we combine them into instructors ID, the child's name and the phone number we will get one relationship where there is no violation of BCNF it is in BCNF but, we have MVDs, we have data duplication. So, you get to know that this person's child is David and William, but you record it twice this phone number you record twice and more and more you have reexplored. Different examples and perspective we just.

(Refer Slide Time: 09:12)

MVD: Definition

- Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency** $\alpha \twoheadrightarrow \beta$ holds on R if in any legal relation $r(R)$, for all pairs of tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that:
 - $t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$ ✓
 - $t_3[\beta] = t_1[\beta]$
 - $t_3[R - \beta] = t_2[R - \beta]$ ✓
 - $t_4[\beta] = t_2[\beta]$
 - $t_4[R - \beta] = t_1[R - \beta]$

Example: A relation of university courses, the books recommended for the course, and the lecturers who will be teaching the course:

- $course \twoheadrightarrow book$
- $course \twoheadrightarrow lecturer$

Test: $course \twoheadrightarrow book$

Course	Book	Lecturer	Tuples
AHA	Siberschatz	John D	t1
AHA	Nederpelt	William M	t2
AHA	Siberschatz	William M	t3
AHA	Nederpelt	John D	t4
AHA	Siberschatz	Christian G	
AHA	Nederpelt	Christian G	
OSO	Siberschatz	John D	
OSO	Siberschatz	William M	

MVD: Definition

Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency** $\alpha \twoheadrightarrow \beta$ holds on R if in any legal relation $r(R)$, for all pairs of tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that:

$$\begin{aligned} t_1[\alpha] &= t_2[\alpha] = t_3[\alpha] = t_4[\alpha] \\ t_3[\beta] &= t_1[\beta] \\ t_4[\beta] &= t_2[\beta] \end{aligned}$$

Example: A relation of university courses, the books recommended for the course, and the lecturers who will be teaching the course:

- $\text{course} \twoheadrightarrow \text{book}$
- $\text{course} \twoheadrightarrow \text{lecturer}$

Test: $\text{course} \twoheadrightarrow \text{book}$

Course	Book	Lecturer	Tuples
AHA	Siberschatz	John D	11
AHA	Nederpelt	William M	12
AHA	Siberschatz	William M	13
AHA	Nederpelt	John D	14
AHA	Siberschatz	Christian G	
AHA	Nederpelt	Christian G	
OSO	Siberschatz	John D	
OSO	Siberschatz	William M	

MVD: Definition

Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency** $\alpha \twoheadrightarrow \beta$ holds on R if in any legal relation $r(R)$, for all pairs of tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that:

$$\begin{aligned} t_1[\alpha] &= t_2[\alpha] = t_3[\alpha] = t_4[\alpha] \checkmark \\ t_3[\beta] &= t_1[\beta] \checkmark \\ t_4[\beta] &= t_2[\beta] \checkmark \\ t_3[R - \beta] &= t_1[R - \beta] \checkmark \\ t_4[R - \beta] &= t_2[R - \beta] \checkmark \end{aligned}$$

Example: A relation of university courses, the books recommended for the course, and the lecturers who will be teaching the course:

- $\text{course} \twoheadrightarrow \text{book}$
- $\text{course} \twoheadrightarrow \text{lecturer}$

Test: $\text{course} \twoheadrightarrow \text{book}$

Course	Book	Lecturer	Tuples
AHA	Siberschatz	John D	11
AHA	Nederpelt	William M	12
AHA	Siberschatz	William M	13
AHA	Nederpelt	John D	14
AHA	Siberschatz	Christian G	
AHA	Nederpelt	Christian G	
OSO	Siberschatz	John D	
OSO	Siberschatz	William M	

So, having understood the situation let us now formalize it like we did for the functional dependency. So, what you say is if R is a relational schema and α and β are subsets of R , then a multivalued dependency as we write holds on R if any legal relation so that is that is always a common statement. The for all pair of tuples t_1 and t_2 that match on α . In functional dependency, what do you say?

If they match on α , they will match on β but it is but now it is not that it is a lot more explosive. If they match on α then there will exist two more tuple t_3 and t_4 such that all of them match on α , t_3 and t_1 match on β , $t_3[R - \beta]$ that is the remaining attributes match with t_1 's $[R - \beta]$, t_4 matches with t_2 on β and so on. All of these, looks complicated if you write down in mathematical form, but it is as such not that the example will help.

So, here is an example where I have two MVDs, course multi determines book and course multi determines lecturer. A course may have multiple so, saying that a course may have multiple books recommended for it and the course may be instructed by more than one lecturer. So, if you look into what is written here as t3 t4, what you'll find that basically what you have here you have two values each Silbertschatz, and Nederpelt.

You have two values here, John and William, but what you will get is you will get all combinations of the Cartesian product of them and that that precisely is this definition. So, if you take these as say, if if you take that the course multi determines this. So, this is t1 this is t2. So, they match on alphas. So, you will get two more t3 t4 who match on alpha, t3 beta should equate t1 beta.

What is t3 beta? t3, this book t3 beta should equate t1 beta t3[R-beta] will equate t3[R-beta] will equate t2[R-beta] like this. It is nothing but actually saying that all the Cartesian product items will come in here. So, that is the formal mathematical definition which can be used to reason on the MVDs. And naturally, as you have an MVD on course to book you have another MVD on course to lecture that will always be there.

(Refer Slide Time: 12:49)

MVD: Example

- Let R be a relation schema with a set of attributes that are partitioned into 3 nonempty subsets Y, Z, W .
- We say that $Y \twoheadrightarrow Z$ (Y **multidetermines** Z) if and only if for all possible relations $r(R)$

$$\begin{aligned} & \langle y_1, z_1, w_1 \rangle \in r \text{ and } \langle y_1, z_2, w_2 \rangle \in r \\ & \text{then} \\ & \langle y_1, z_1, w_2 \rangle \in r \text{ and } \langle y_1, z_2, w_1 \rangle \in r \end{aligned}$$
- Note that since the behavior of Z and W are identical it follows that $Y \twoheadrightarrow Z$ if $Y \twoheadrightarrow W$.

MVD: Example

- Let R be a relation schema with a set of attributes that are partitioned into 3 nonempty subsets. Y, Z, W
- We say that $Y \twoheadrightarrow Z$ (Y **multidetermines** Z) if and only if for all possible relations $r(R)$ $\langle y_1, z_1, w_1 \rangle \in r$ and $\langle y_1, z_2, w_2 \rangle \in r$ then $\langle y_1, z_1, w_2 \rangle \in r$ and $\langle y_1, z_2, w_1 \rangle \in r$
- Note that since the behavior of Z and W are identical it follows that $Y \twoheadrightarrow Z$ if $Y \twoheadrightarrow W$



The diagram shows a large circle labeled 'R' representing a relation. Inside the circle, there are three smaller circles labeled 'Y', 'Z', and 'W'. Arrows point from 'Y' to 'Z' and from 'Y' to 'W'. A double-headed arrow connects 'Z' and 'W', indicating they are identical in behavior. The circles are arranged in a triangular pattern within 'R'.

So, if we have a relational schema with a set of attributes, which is partitioned into three non-empty subsets Y, Z and W , there is partition. So, together it will form R and they are disjointed. Then you say that, if Y multi determines Z if for all possible relations, if y_1, z_1, w_1 and y_1, z_2, w_2 both exist in R , then the Cartesian product combinations will also exist y_1, z_1, w_2 and y_1, z_2, w_1 that is and we decide in a different way here, this is a partition of R .

So, since it is so, so, you can clearly see that, if you just have such a partition, then what it will mean that if you have a MVD from Y to Z , then you will also have an MVD to Y to W . So, what we say is if you have an MVD to Y to say X some subset of R , then you will also have an MVD to Y to $R-X$ that is the partition. So, that is what you observe in this that is that is what you saw in the previous one also. We had course to book there are three attributes course, book and lecturer, a course to book MVD and therefore I have a course to lecture MVD or vice versa. So, this is a generalization of that notion.

(Refer Slide Time: 14:33)

MVD: Example (2)

- In our example:
 $ID \twoheadrightarrow child_name$
 $ID \twoheadrightarrow phone_number$
- The above formal definition is supposed to formalize the notion that given a particular value of $Y(ID)$ it has associated with it a set of values of $Z (child_name)$ and a set of values of $W (phone_number)$, and these two sets are in some sense independent of each other.
- Note:
 - If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$
 - Indeed we have (in above notation) $Z_1 = Z_2$
The claim follows.

MVD: Example (2)

- In our example:
 $ID \twoheadrightarrow child_name$
 $ID \twoheadrightarrow phone_number$
- The above formal definition is supposed to formalize the notion that given a particular value of $Y(ID)$ it has associated with it a set of values of $Z (child_name)$ and a set of values of $W (phone_number)$, and these two sets are in some sense independent of each other.
- Note:
 - If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$
 - Indeed we have (in above notation) $Z_1 = Z_2$
The claim follows.

So again, similar statements for ID and child name. If you have ID to child name you will also have child name to ID to child name will also have ID to phone number. Now, if we say that Y functionally determines Z, Y functionally determines Z, then we will always have that Y multi determined Z, because all that is happening is that the two sets are actually identical, you can consider them to be identical.

Here you are having Z you having W, they are basically identical, just two partitions. So, this is a golden rule to understand that if I have a functional dependency, I always have a corresponding multivalued dependency, but not the reverse the reverse is not true from the multivalued dependency I cannot come to functional dependence but this way it is true.

(Refer Slide Time: 15:48)

The screenshot shows a presentation slide titled "MVD: Use" from IIT Madras. The slide content is as follows:

- We use multivalued dependencies in two ways:
 - a) To test relations to **determine** whether they are legal under a given set of functional and multivalued dependencies
 - b) To specify **constraints** on the set of legal relations. We shall thus concern ourselves only with relations that satisfy a given set of functional and multivalued dependencies.
- If a relation r fails to satisfy a given multivalued dependency, we can construct a relations r' that does satisfy the multivalued dependency by adding tuples to r .

The slide is part of a presentation titled "Module 29: Functional Dependencies" and is the 11th slide. A video feed of a presenter is visible in the bottom right corner of the slide.

So, the how do you I mean what way do you use that possibly two ways one is test relations to determine whether they are legal under a set of functional and multivalued dependency and to specify constraints on the legal relation. Because if a relation fails to satisfy multivalued dependency, then we can always add the because those tuples are valid. But if an instance those are not there, then we can always add those tuples to satisfy the multivalued dependency.

(Refer Slide Time: 16:29)

Page 40 of 41

MVD: Theory

Module 29
Parthiv Prabhu Das

	Name	Rule
C-	Complementation	If $X \twoheadrightarrow Y$, then $X \twoheadrightarrow (R - (X \cup Y))$.
A-	Augmentation	If $X \twoheadrightarrow Y$ and $W \supseteq Z$, then $WX \twoheadrightarrow YZ$.
T-	Transitivity	If $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$, then $X \twoheadrightarrow (Z - Y)$.
	Replication	If $X \twoheadrightarrow Y$, then $X \twoheadrightarrow Y$ but the reverse is not true.
	Coalescence	If $X \twoheadrightarrow Y$ and there is a W such that $W \cap Y$ is empty, $W \twoheadrightarrow Z$ and $Y \supseteq Z$, then $X \twoheadrightarrow Z$.

- A MVD $X \twoheadrightarrow Y$ in R is called a trivial MVD is
 - Y is a subset of X ($X \supseteq Y$) or
 - $X \cup Y = R$. Otherwise, it is a non trivial MVD and we have to repeat values redundantly in the tuples.

Parthiv Prabhu Das

Page 41 of 41

MVD: Theory

Module 29
Parthiv Prabhu Das

	Name	Rule
C-	Complementation	If $X \twoheadrightarrow Y$, then $X \twoheadrightarrow (R - (X \cup Y))$.
A-	Augmentation	If $X \twoheadrightarrow Y$ and $W \supseteq Z$, then $WX \twoheadrightarrow YZ$.
T-	Transitivity	If $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$, then $X \twoheadrightarrow (Z - Y)$.
	Replication	If $X \twoheadrightarrow Y$, then $X \twoheadrightarrow Y$ but the reverse is not true.
	Coalescence	If $X \twoheadrightarrow Y$ and there is a W such that $W \cap Y$ is empty, $W \twoheadrightarrow Z$ and $Y \supseteq Z$, then $X \twoheadrightarrow Z$.

- A MVD $X \twoheadrightarrow Y$ in R is called a trivial MVD is
 - Y is a subset of X ($X \supseteq Y$) or
 - $X \cup Y = R$. Otherwise, it is a non trivial MVD and we have to repeat values redundantly in the tuples.

Parthiv Prabhu Das

Page 42 of 41

MVD: Theory

Module 29
Parthiv Prabhu Das

	Name	Rule
C-	Complementation	If $X \twoheadrightarrow Y$, then $X \twoheadrightarrow (R - (X \cup Y))$.
A-	Augmentation	If $X \twoheadrightarrow Y$ and $W \supseteq Z$, then $WX \twoheadrightarrow YZ$.
T-	Transitivity	If $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$, then $X \twoheadrightarrow (Z - Y)$.
	Replication	If $X \twoheadrightarrow Y$, then $X \twoheadrightarrow Y$ but the reverse is not true.
	Coalescence	If $X \twoheadrightarrow Y$ and there is a W such that $W \cap Y$ is empty, $W \twoheadrightarrow Z$ and $Y \supseteq Z$, then $X \twoheadrightarrow Z$.

- A MVD $X \twoheadrightarrow Y$ in R is called a trivial MVD is
 - Y is a subset of X ($X \supseteq Y$) or
 - $X \cup Y = R$. Otherwise, it is a non trivial MVD and we have to repeat values redundantly in the tuples.

Parthiv Prabhu Das

Now, likely we had Armstrong's actions, we have similar inferencing rules for MVDs as well. For example, of that, a special case of this is what we have seen, in general it is if R is X multi determines Y then X multi determines R- X union Y. So, you are creating that partition. Augmentation rule of functional dependencies gets stronger because now you do not need to augment with the same set on both sides, but left side maybe augmented with a larger set and the right side could be a smaller set as well.

The transitivity takes a different look as you can see, so, these you will have to, these are not difficult, you will have to sit down take examples or take the definition and get convinced that these rules are what hold. So, if X multi determines Y and Y multi determine Z then X multi determines Z-Y. Earlier it was Z alone. Now, it is Z-Y. Similarly, for replication this we have just said, in another coalescence.

So, these are different inferencing rules like the Armstrong's actions we had for multi valued dependencies. The notion of triviality also changes earlier we said that a functional dependency is trivial, trivial, if the left-hand, right-hand side is a subset of the left-hand side. But now that holds but in addition, it is also trivial if the union of the left-hand right-hand side is a whole set. Because of you already have this kind of rule. So, by that you will say that X union Y if it is R, then it is then X multi determines Y is a trivial multivalued dependency.

(Refer Slide Time: 18:43)

The screenshot shows a presentation slide titled "MVD: Theory (2)" from IIT Madras. The slide content is as follows:

- From the definition of multivalued dependency, we can derive the following rule:
 - If $\alpha \twoheadrightarrow \beta$, then $\alpha \twoheadrightarrow \beta$
 That is, every functional dependency is also a multivalued dependency
- The **closure** D^+ of D is the set of all functional and multivalued dependencies logically implied by D.
 - We can compute D^+ from D, using the formal definitions of functional dependencies and multivalued dependencies.
 - We can manage with such reasoning for very simple multivalued dependencies, which seem to be most common in practice
 - For complex dependencies, it is better to reason about sets of dependencies using a system of inference rules

So, what we now get into actually using it in the normalization. So, we have the first following rule that if alpha determines beta than alpha multi determines beta. And in the same way that we did the closure of functional dependencies, we can now do the closure of all

functional as well as multivalued dependencies. We can compute them by using the formal definition.

In many cases, the situations of multiple values are not that complex. So, we can manage by doing simple reasoning, as we have been doing now. But if the situations are complex, then we will have to use inferencing rules that I just showed, and reason that to get to the closure of a set of functional as well as multivalued dependency and the concepts remaining concepts remain all the same. It is just that the dependency is being being relaxed in a different way.

(Refer Slide Time: 19:56)

The image shows two slides from a presentation by Partha Pratim Das at IIT Madras, titled 'Database Management Systems'.

Slide 1: Decomposition to 4NF

Decomposition to 4NF

Slide 2: Fourth Normal Form

- A relation schema R is in **4NF** with respect to a set D of functional and multivalued dependencies if for all multivalued dependencies in D^+ of the form $\alpha \twoheadrightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following hold:
 - $\alpha \twoheadrightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$ or $\alpha \cup \beta = R$) ✓
 - α is a superkey for schema R ✓
- If a relation is in 4NF it is in BCNF

Handwritten notes on the right side of the slide:

- 1NF ✓
- 2NF ✓
- 3NF ✓
- BCNF/3NF ✓
- 4NF

So, now, we will define and show the decomposition in the presence of multivalued dependency. So, like BCNF, 3NF and so on, we say that a relational schema is in 4NF. So, we have we have 1NF we have 2NF we have 3NF we have BCNF which many people call as

3.5NF and now, we have 4NF. Atomic value, partial dependency, transitive dependency and this is superkey restriction.

So, what do you how you define 4NF? Say that with respect to the set of functional and multivalued dependencies if for all multivalued dependencies in D plus that is a closure set of the form alpha multi determines beta at least one of the two will have to hold. One is either it is a trivial MVD or is a superkey, superkey of the schema.

So, only thing that you are injecting is a notion of multivalued dependency but at the end, the definition of 4NF is pretty much like BCNF and it is it will be quite obvious to see that if a relation is in 4NF, it is necessarily in BCNF because every functional dependency is a multivalued dependency as well. So, again the conditions are, every multivalued dependency is either trivial or the left-hand side is the superkey of the schema.

(Refer Slide Time: 22:08)

Restriction of Multivalued Dependencies

- The restriction of D to R_i is the set D_i consisting of
 - All functional dependencies in D^+ that include only attributes of R_i ✓
 - All multivalued dependencies of the form $\alpha \twoheadrightarrow (\beta \cap R_i)$ ✓
where $\alpha \subseteq R_i$ and $\alpha \twoheadrightarrow \beta$ is in D^+

Now, to be able to check for different things, you will need to define the restriction in the same way. So, if you have decomposed a relation to R_i , then you have to restrict D the set of functional and multivalued dependencies to D_i which has all the functional dependencies in D plus that include only attributes of R_i this is what we have been doing earlier. And this is the addition that it will have all multivalued dependencies of the form beta in alpha multi determines beta intersection R_i , where alpha is in R_i , and this is in D plus. That is quite clear.

(Refer Slide Time: 23:17)

4NF Decomposition Algorithm

- a) For all dependencies $A \twoheadrightarrow B$ in D^+ , check if A is a superkey
 - By using attribute closure
- b) If not, then
 - Choose a dependency in F^+ that breaks the 4NF rules, say $A \twoheadrightarrow B$
 - Create $R_1 = A B$
 - Create $R_2 = (R - (B - A))$
 - Note that: $R_1 \cap R_2 = A$ and $A \twoheadrightarrow AB (= R_1)$, so this is lossless decomposition
- c) Repeat for R_1 , and R_2
 - By defining D_1^+ to be all dependencies in F that contain only attributes in R_1
 - Similarly D_2^+

Now, if a relationship is not in 4NF, we will again do the same we will decompose to make it into 4NF. And it goes exactly like what we did for BCNF. So, we will check using attribute closure defined in the same way as we did for functional dependency. Whether A multi determines B is in the closure set if it is not, if this from this closure set, we will know whether it is a, A is a super key or not.

If A is not a super key, then we will have to create a decomposition and in the same way, we will have A union B in one decomposition and A minus B minus in other decomposition. Mind you the consequent observation remains same that the intersection is A and therefore you will have a lossless join. There is now no problem with that. So, this is exactly like the BCNF except that you are doing the closure in terms of the multivalued dependencies.

Now, after you have got R_1 and R_2 , we have to check each if they are in 4NF. And for checking that we have to create those restrictions D_1 and D_2 and check within D_1 , the closure of D_1 and closure of D_2 and if any one or both of them are not in multivalued or not in 4NF, then I will have to again go forward and decompose it further.

(Refer Slide Time: 25:13)

4NF Decomposition Algorithm

```

result := {R};
done := false;
compute  $D^+$ ;
Let  $D_i$  denote the restriction of  $D^+$  to  $R_i$ ;
while (not done)
  if (there is a schema  $R_i$  in result that is not in 4NF) then
    begin
      let  $\alpha \twoheadrightarrow \beta$  be a nontrivial multivalued dependency that holds
      on  $R_i$  such that  $\alpha \rightarrow R_i$  is not in  $D_i$ , and  $\alpha \cap \beta = \phi$ ;
      result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );
    end
  else done := true;
Note: each  $R_i$  is in 4NF, and decomposition is lossless-join

```

So, formally said this is the pseudocode of the algorithm, which you can go through and understand in steps, but what you have said is the basic process. So, you have the restrictions being computed that is the first thing, restrictions being computed, and then you are doing the checking. And based on that you do the partitioning of the relational schema.

(Refer Slide Time: 25:43)

Example of 4NF Decomposition

• Example:

- Person_Modify(Man(M), Phones(P), Dog_Likes(D), Address(A))
- FDs:
 - FD1: Man $\twoheadrightarrow$ Phones
 - FD2: Man $\twoheadrightarrow$ Dogs_Like
 - FD3: Man $\rightarrow$ Address
- Key = MPD
- All dependencies violate 4NF

Post Normalization

Person_Phones, Person_DogsLikes, Person_Address

In the above relations for both the MVD's - $X^* \text{ is Man}$, which is again not the super key, but as $X \cup Y = R$ i.e. (Man & Phones) together make the relation. So, the above MVD's are trivial and in FD 3, Address is functionally dependent on Man, where Man is the key in Person_Address, hence all the three relations are in 4NF.

Man(M)	Phones(P)	Dogs_Likes(D)	Address(A)
M1	P1	D1	49-ABC,Bhiwani(HR)
M1	P2	D2	49-ABC,Bhiwani(HR)
M2	P3	D2	36-XYZ,Rohatki(HR)
M1	P1	D2	49-ABC,Bhiwani(HR)
M1	P2	D1	49-ABC,Bhiwani(HR)

So, now we have learned how to create 4NF so, let us look at an example. So, let us go back to our person relational schema, all that I have done is I have included another attribute, which is not multivalued, it is a single valued, a person has one address, multiple phones, multiple dog likes. So, there is a I mean, it is written as FD this basically dependencies.

So, first one is an MVD, man multi determines phones, can have multiple phones, can have multiple likes to dogs man multi determines dog and has a unique address naturally given this all the three attributes are forming the key. So, here as you can see that all dependencies violate 4NF because none of them are key on the left-hand side and none of them are trivial. So, what we would do?

We would use this violation and create a decomposition here which is person phone which is man and phone, the remaining will also so this is what will remain is person, dog like, address that will also violate. So, we will take this, create this and we will be left with the last one. So, here in this relation, we have one multivalued dependency in the restriction here we have another multivalued dependency in the restriction, here we have the functional dependency and the restriction you can see the notations here.

FD1, FD2, FD3. So, with this normalization, the entire data schema comes into fourth normal form. So, it has minimal amount of redundancy, it naturally through the process has lossless joint and as it so happens that all the 3, FD or MVD are shakeable in the respective decomposed relation. Therefore, this is also dependency preserving decomposition into 4NF incidentally.

(Refer Slide Time: 28:29)

Example of 4NF Decomposition

- $R = (A, B, C, G, H, I)$
- $F = \underline{A} \twoheadrightarrow B$
- $B \twoheadrightarrow HI$
- $\underline{CG} \twoheadrightarrow H$
- R is not in 4NF since $A \twoheadrightarrow B$ and A is not a superkey for R
- Decomposition
 - $R_1 = (A, B)$ ✓ (R_1 is in 4NF)
 - $R_2 = (A, C, G, H, I)$ ✓ (R_2 is not in 4NF, decompose into R_3 and R_4)
 - $R_3 = (C, G, H)$ ✓ (R_3 is in 4NF)
 - $R_4 = (A, C, G, I)$ ✓ (R_4 is not in 4NF, decompose into R_5 and R_6)
 - $A \twoheadrightarrow B$ and $B \twoheadrightarrow HI \rightarrow A \twoheadrightarrow HI$, (MVD transitivity), and
 - and hence $A \twoheadrightarrow I$ (MVD restriction to R_4)
 - $R_5 = (A, I)$ ✓ (R_5 is in 4NF)
 - $R_6 = (A, C, G)$ ✓ (R_6 is in 4NF)

Example of 4NF Decomposition

- $R = (A, B, C, G, H, I)$
 $F = A \twoheadrightarrow B$
 $B \twoheadrightarrow HI$
 $CG \twoheadrightarrow H$
- R is not in 4NF since $A \twoheadrightarrow B$ and A is not a superkey for R
- Decomposition
 - a) $R_1 = (A, B)$ (R_1 is in 4NF)
 - b) $R_2 = (A, C, G, H, I)$ (R_2 is not in 4NF, decompose into R_3 and R_4)
 - c) $R_3 = (C, G, H)$ (R_3 is in 4NF)
 - d) $R_4 = (A, C, G, I)$ (R_4 is not in 4NF, decompose into R_5 and R_6)
 - $A \twoheadrightarrow B$ and $B \twoheadrightarrow HI \rightarrow A \twoheadrightarrow HI$, (MVD transitivity), and
 - and hence $A \twoheadrightarrow I$ (MVD restriction to R_4)
 - e) $R_5 = (A, I)$ (R_5 is in 4NF)
 - f) $R_6 = (A, C, G)$ (R_6 is in 4NF)

This is a longer example where we have started with considering this A, B, I mean A multi determines B which is which violates, so I have A, B and I have the remaining A, C, G, H, I . Now, this is in 4NF clearly but A, C, G, H, I is not in 4NF because it will have C, G multi determines H . As we have that so we again partition C, G, H and this C, G, H is in 4NF but A, C, G, I is not.

To see that A, C, G, I is not we have to actually get into the closure of the set of functional multivalued dependencies. And you can see using the MVD transitivity that there is a multivalued dependency A multi determining I which is available in the restriction R_4 . So, naturally that will get violated. So, we have to partition R_4 again, I get A, I , I get A, C, G . So finally, R_1, R_3, R_5, R_6 these four together give me a 4NF decomposition of the original relation.

(Refer Slide Time: 30:11)

Module Summary

- Understood multi-valued dependencies to handle attributes that can have multiple values
- Learnt Fourth Normal Form and decomposition to 4NF

Slides used in this presentation are borrowed from <http://db-book.com/> with kind permission of the authors.
Edited and new slides are marked with "PPD".

Database Management Systems Partha Pratim Das 29/22

So, in this module, as we conclude, we have understood the additional redundancy factors in the design of relational schema and multi valued attributes situation we have tried to address and seen that I can all we can take a schema into 4NF through decomposition, ensuring lossless joint, it may or may not preserve dependencies. In reality, I would say that it is much less frequent that you want users the 4NF because the prevalence of multiple valued attributes are not that huge.

Or maybe you will take care of those but they are very typical like phone number like address and so on. So, we will take care of those in that way. But this is the general founding theory for this and it is also that this does not mean that 4NF gets rid of all kinds of redundancies. There are other types of redundancies like joint redundancy and so on for each 5NF and so on so forth.

There are several other normal forms higher and higher, which removes more and more redundancies. But we will stop here on the theory of design of relational databases, because those are not very frequently used. Thank you very much for your attention and we meet in the next module.